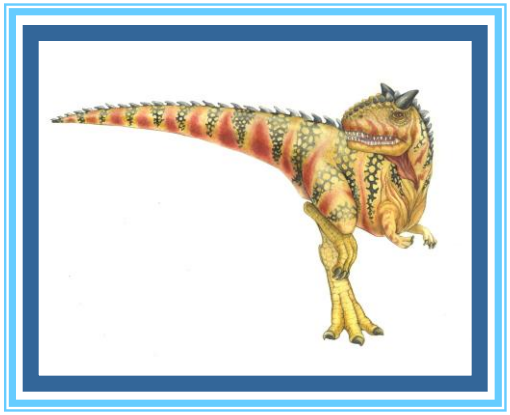
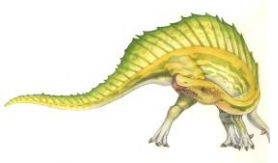
Chapter 9: Virtual Memory



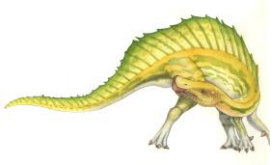
Chapter 9: Virtual Memory

- Background
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing



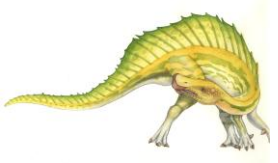
Background

- Code needs to be in memory to execute, but entire program rarely used
  - Error code, unusual routines, large data structures
- Entire program code not needed at same time
- Consider ability to execute partially-loaded program
  - Program no longer constrained by limits of physical memory
  - Each program takes less memory while running -> more programs run at the same time
    - Increased CPU utilization and throughput with no increase in response time or turnaround time
  - Less I/O needed to load or swap programs into memory -> each user program runs faster



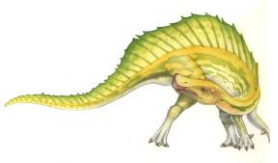
Background (Cont.)

- **Virtual memory** – separation of user logical memory from physical memory
  - Only part of the program needs to be in memory for execution
  - Logical address space can therefore be much larger than physical address space
  - Allows address spaces to be shared by several processes
  - Allows for more efficient process creation
  - More programs running concurrently
  - Less I/O needed to load or swap processes

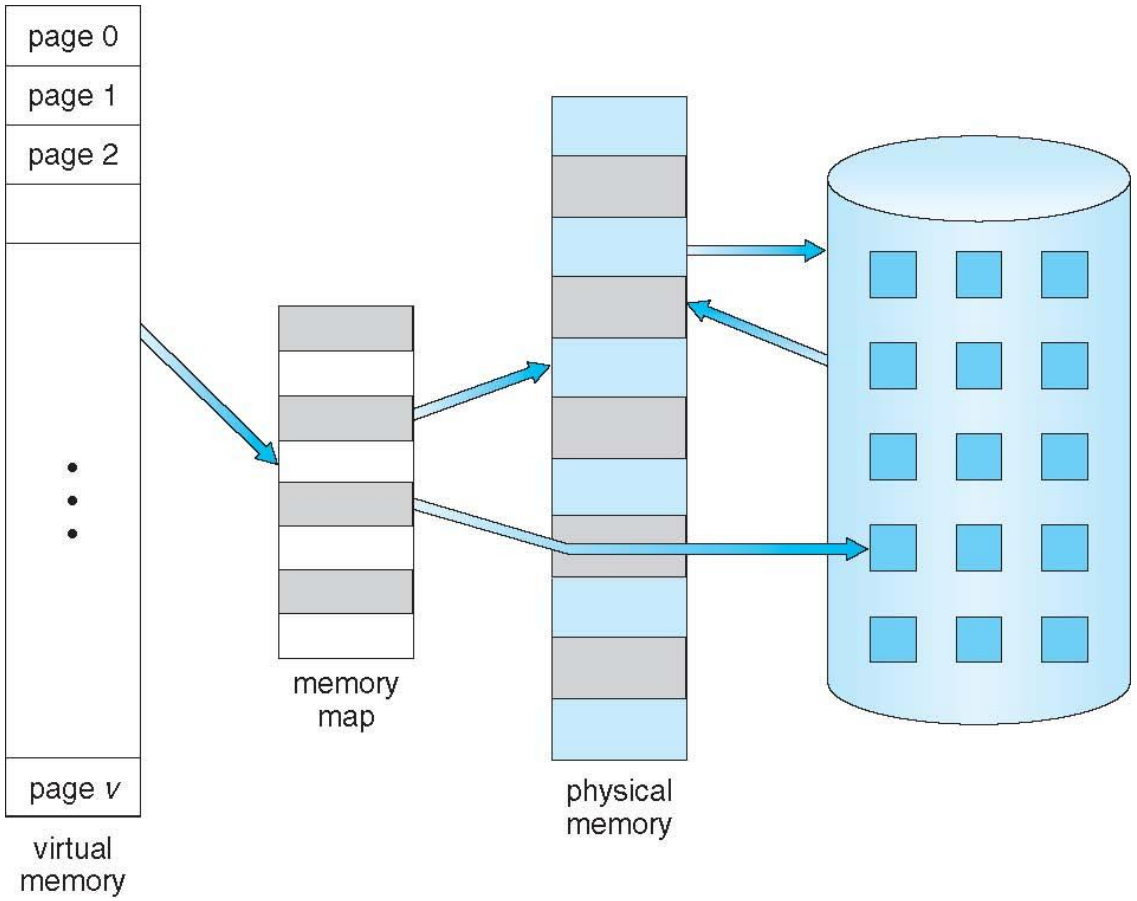


Background (Cont.)

- **Virtual address space** – logical view of how process is stored in memory
  - Usually start at address 0, contiguous addresses until end of space
  - Meanwhile, physical memory organized in page frames
  - MMU must map logical to physical
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation

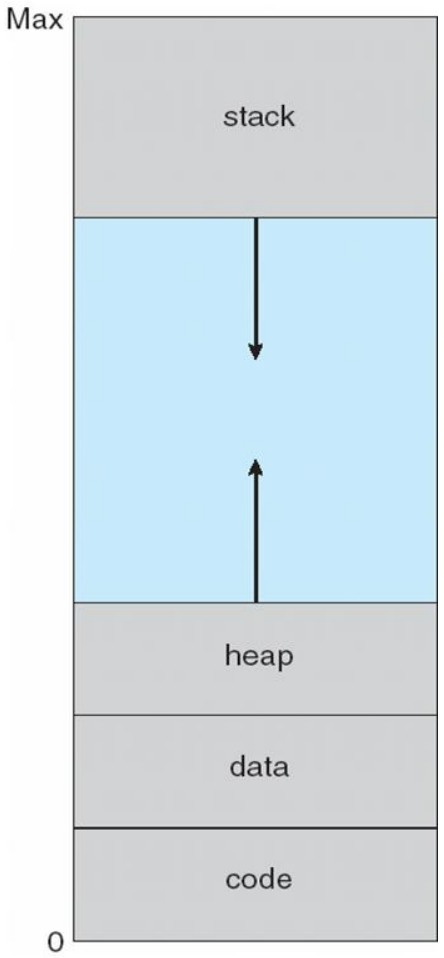


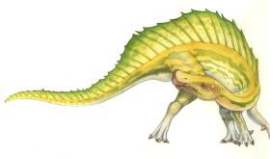
Virtual Memory That is Larger Than Physical Memory



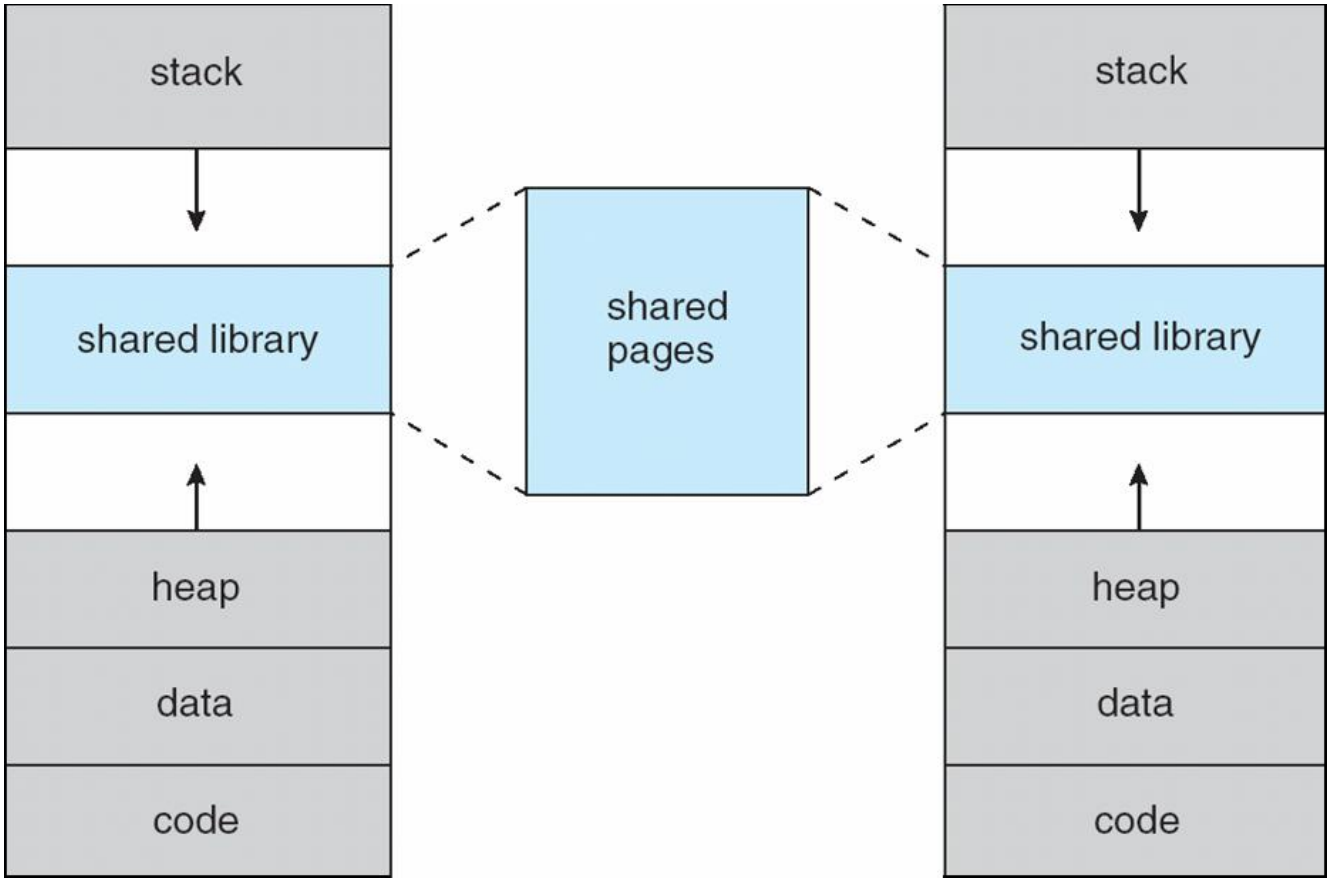
Virtual-address Space

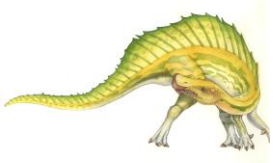
- Usually design logical address space for stack to start at Max logical address and grow "down" while heap grows "up"
  - Maximizes address space use
  - Unused address space between the two is hole
    - No physical memory needed until heap or stack grows to a given new page
- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during `fork()`, speeding process creation





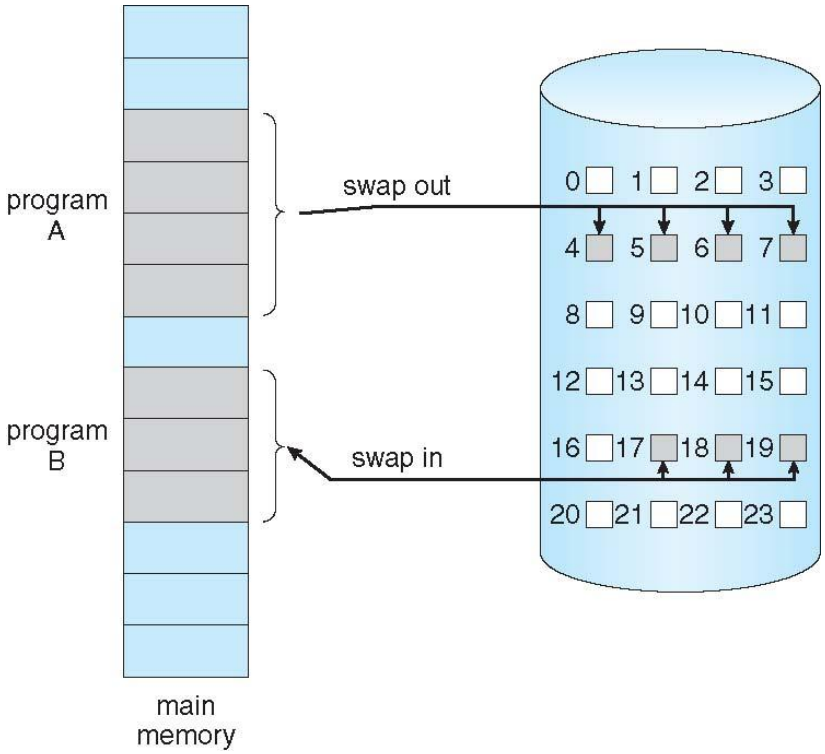
Shared Library Using Virtual Memory

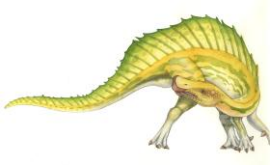




Demand Paging

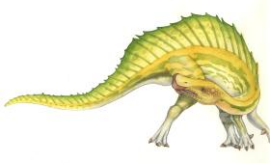
- Could bring entire process into memory at load time
- Or bring a page into memory only when it is needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- Similar to paging system with swapping (diagram on right)
- Page is needed ⇒ reference to it
  - invalid reference ⇒ abort
  - not-in-memory ⇒ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
  - Swapper that deals with pages is a **pager**





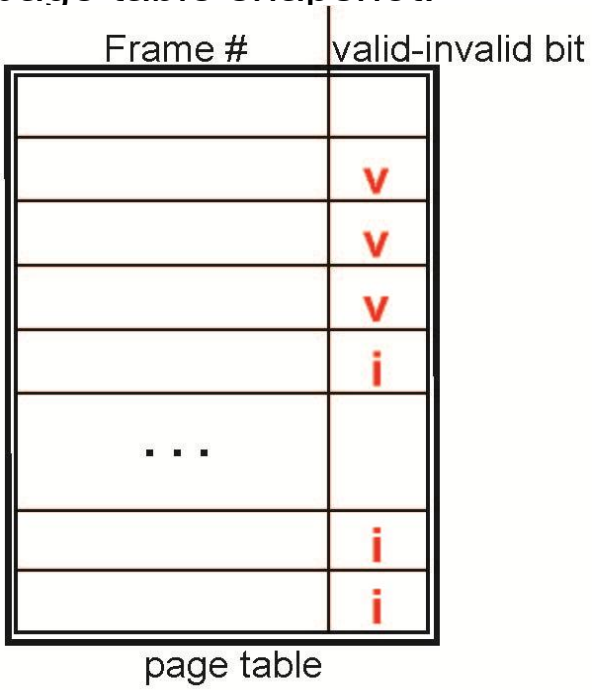
Basic Concepts

- With swapping, pager guesses which pages will be used before swapping out again
- Instead, pager brings in only those pages into memory
- How to determine that set of pages?
  - Need new MMU functionality to implement demand paging
- If pages needed are already **memory resident**
  - No difference from non demand-paging
- If page needed and not memory resident
  - Need to detect and load the page into memory from storage
    - Without changing program behavior
    - Without programmer needing to change code



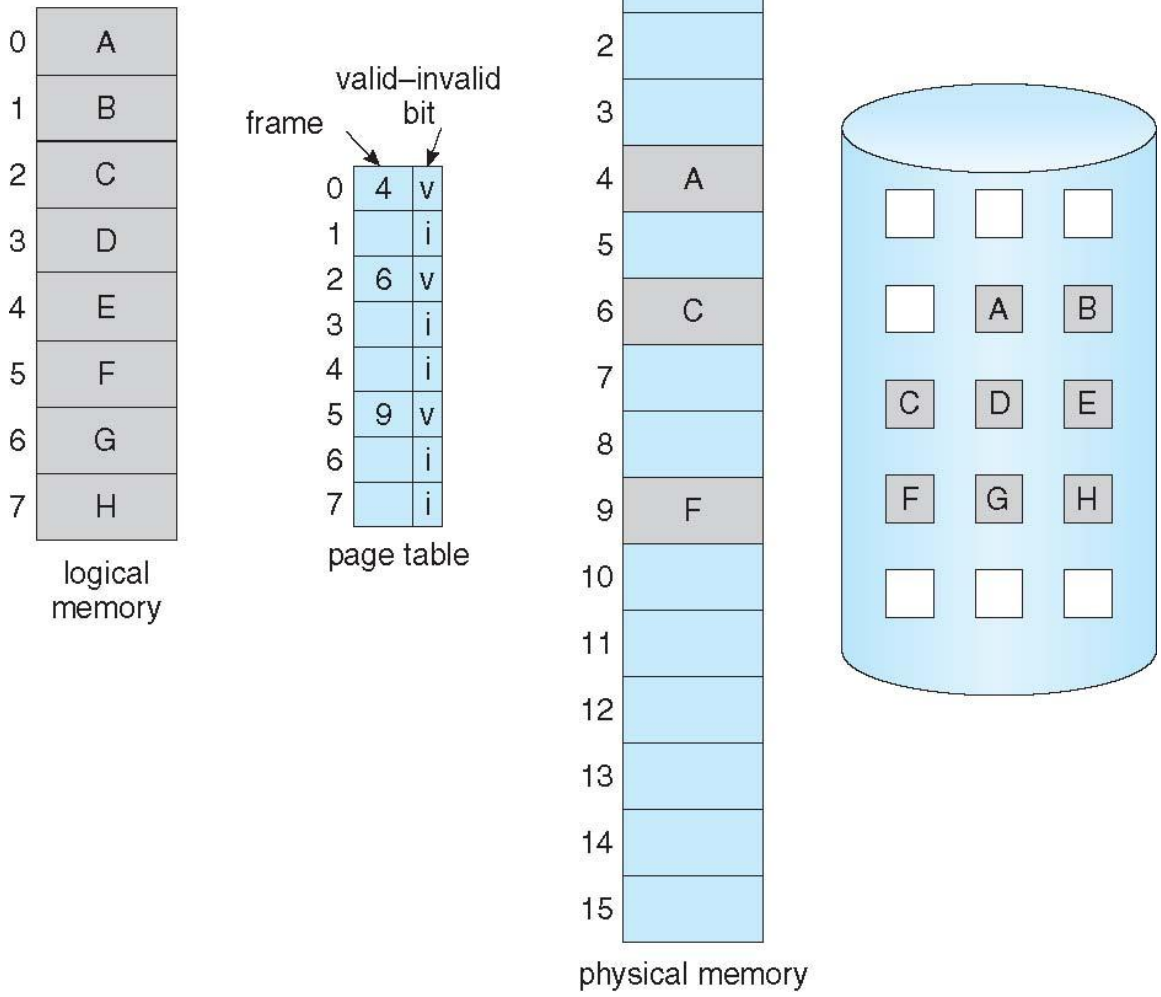
Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** ⇒ in-memory – **memory resident**, **i** ⇒ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:



- During MMU address translation, if valid–invalid bit in page table entry is **i** ⇒ page fault

Page Table When Some Pages Are Not in Main Memory



Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system:

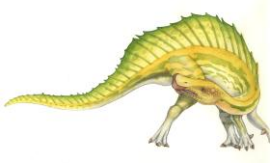
  page fault

- Operating system looks at internal table(usually kept with PCB) to determine whether the reference was a valid or an invalid memory access.
  - Invalid reference ⇒ abort the process
  - Just not in memory

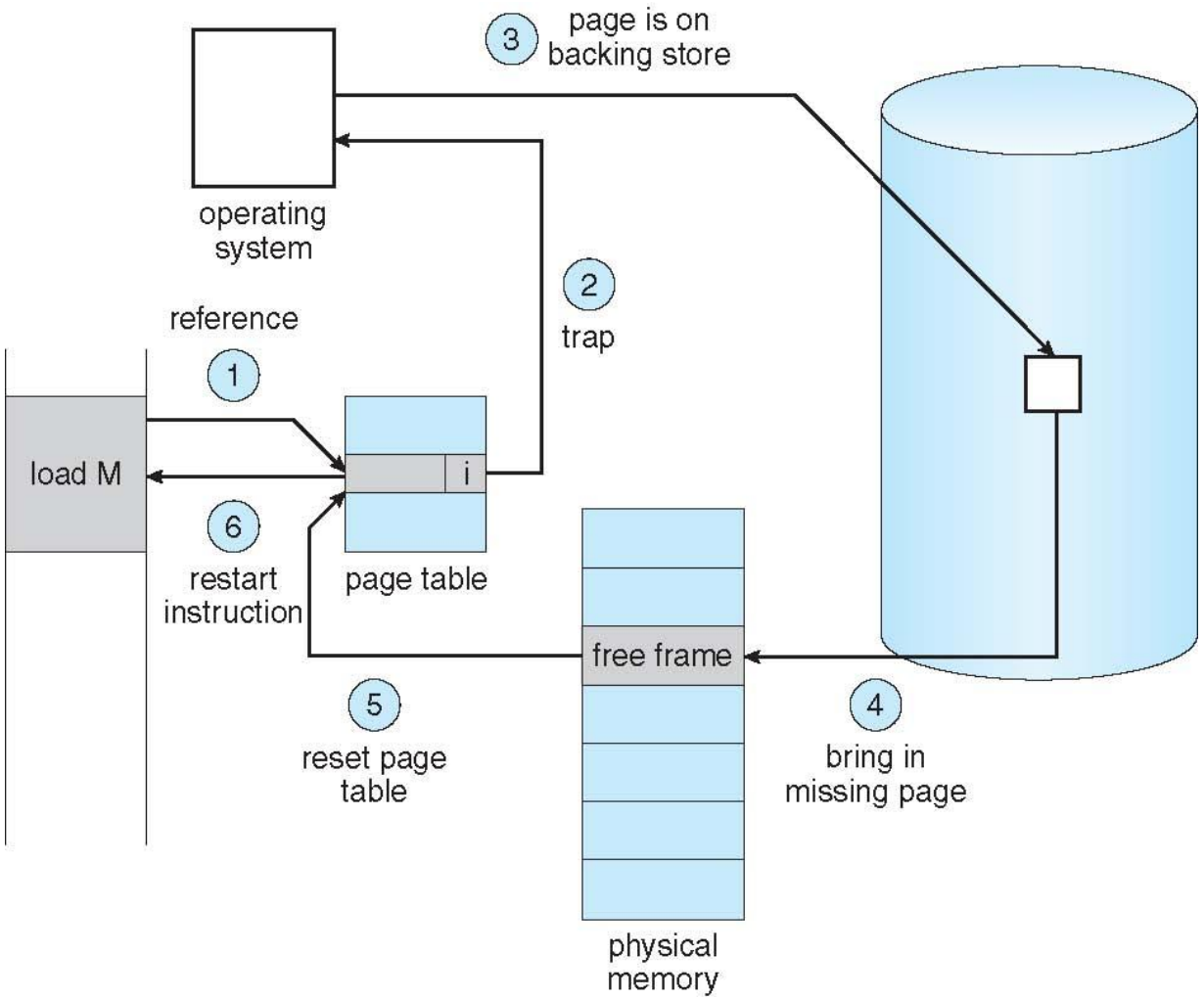
1. Find free frame
2. Swap page into frame via scheduled disk operation

3. modify the internal table kept with the process and the page table to indicate that the page is now in memory. Reset tables to indicate page now in memory
Set validation bit = **v**

4. Restart the instruction that caused the page fault



Steps in Handling a Page Fault

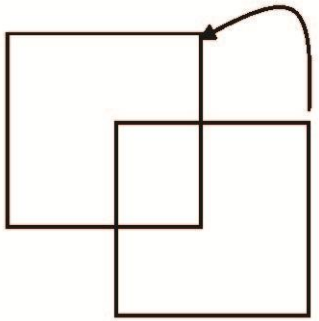


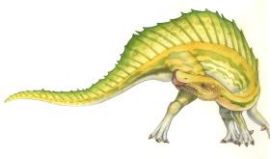
Aspects of Demand Paging

- Extreme case – start process with *no* pages in memory
  - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
- And for every other process pages on first access. After this page is brought into memory, the process continues to execute, faulting as necessary until every page that it needs is in memory.
- At that point, it can execute with no more faults.
- **-Pure demand paging:** : never bring a page into memory until it is required.
- Actually, a given instruction could access multiple pages -> multiple page faults
  - Consider fetch and decode of instruction which adds 2 numbers from memory and stores result back to memory
  - Pain decreased because of **locality of reference**
- Hardware support needed for demand paging
  - Page table with valid / invalid bit
  - Secondary memory (swap device with **swap space**)
  - Instruction restart

Instruction Restart

- restart any instruction after a page fault. Because we save the state (registers, condition code, instruction counter) of the interrupted process when the page fault occurs, we must be able to restart the process in *exactly* the same place and state, except that the desired page is now in memory and is accessible.
- page fault may occur at any memory reference.
  - If occurs on the instruction fetch, restart by fetching the instruction again.
  - If occurs while we are fetching an operand, we must fetch and decode the instruction again and then fetch the operand
- Consider an instruction that could access several different locations
  - block move
  - auto increment/decrement location
  - Restart the whole operation?
    - What if source and destination overlap?

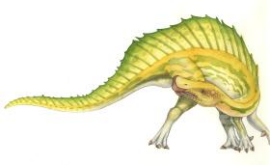




Instruction Restart Contd…

- two solutions

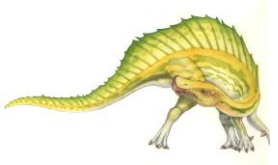
1. microcode computes and attempts to access both ends of both blocks.
   - If a page fault is going to occur, it will happen at this step, before anything is modified.
   - The move can then take place; no page fault can occur, since all the relevant pages are in memory.
2. uses temporary registers to hold the values of overwritten locations.
   - If there is a page fault, all the old values arewritten back into memory before the trap occurs.
   - This action restores memory to its state before the instruction was started, so that the instruction can be repeated



Performance of Demand Paging (Cont.)

- Page Fault Rate $0 \le p \le 1$
  - if $p = 0$ no page faults
  - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)

$$\begin{aligned} EAT = (1 - p) \times & \text{ memory access} \\ & + p \times \text{page fault time (overhead} \\ & \quad + \text{swap page out} \\ & \quad + \text{swap page in)} \end{aligned}$$



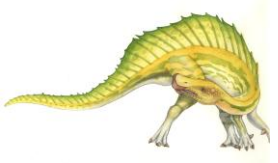
Performance of Demand Paging

- Stages in Demand Paging (worse case)

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
    1. Wait in a queue for this device until the read request is serviced
    2. Wait for the device seek and/or latency time
    3. Begin the transfer of the page to a free frame
6. While waiting, allocate the CPU to some other user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

Performance of Demand Paging

- Three major activities
  - Service the interrupt – careful coding means just several hundred instructions needed
  - Read the page – lots of time
  - Restart the process – again just a small amount of time



Demand Paging Example

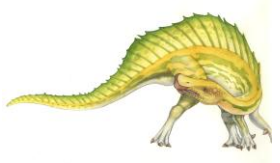
- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- EAT = (1 – p) x 200 + p (8 milliseconds)

  = (1 – p x 200 + p x 8,000,000

  = 200 + p x 7,999,800
- If one access out of 1,000 causes a page fault, then

  EAT = 8.2 microseconds.

  This is a slowdown by a factor of 40!!
- If want performance degradation < 10 percent
  - 220 > 200 + 7,999,800 x p
    20 > 7,999,800 x p
  - p < .0000025
  - < one page fault in every 400,000 memory accesses

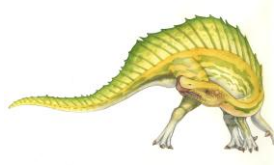


Demand Paging Optimizations

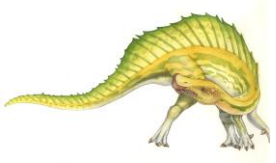
- Swap space I/O faster than file system I/O even if on the same device
  - Swap allocated in larger chunks, less management needed than file system
- Copy entire process image to swap space at process load time
  - Then page in and out of swap space - Used in older BSD Unix
- Demand page in from program binary on disk, but write the pages to swap space as they are replaced.
  - ensures that only needed pages are read from the file system but that all subsequent paging is done from swap space
- Some systems attempt to limit the amount of swap space used through demand paging of binary files.
  - Demand pages for such files are brought directly from the file system.
  - However, when page replacement is called for, these frames can simply be overwritten (because they are never modified), and the pages can be read in from the file system again if needed.
  - the file system itself serves as the backing store.
  - However, swap space must still be used for pages not associated with a file; these pages include the stack and heap for a process.
  - used in several systems, including Solaris and BSD UNIX

Copy-on-Write

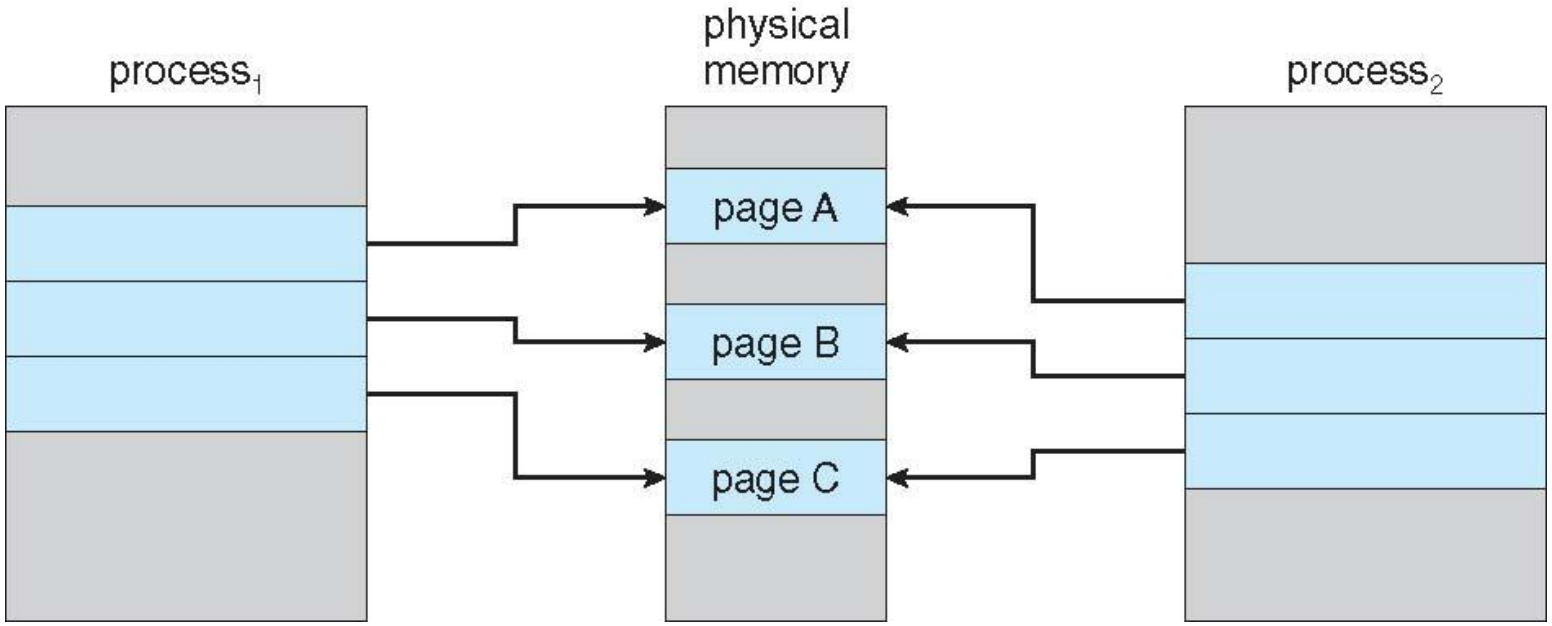
- fork() system call creates a child process that is a duplicate of its parent.
  - worked by creating a copy of the parent's address space for the child, duplicating the pages belonging to the parent.
- many child processes invoke the exec() system call immediately after creation, copying of the parent's address space may be unnecessary. I
- use a technique known as **copy-on-write**,
  - works by allowing the parent and child processes initially to share the same pages.
  - These shared pages are marked as copy-on-write pages,
    - if either process writes to a shared page, a copy of the shared page is created.
- **Copy-on-Write** (COW) allows both parent and child processes to initially ***share*** the same pages in memory
  - If either process modifies a shared page, only then is the page copied

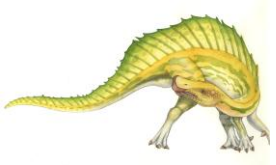


- free pages are allocated from a **pool** of **zero-fill-on-demand** pages
- Zero-fill-on-demand pages have been zeroed-out before being allocated, thus erasing the previous contents

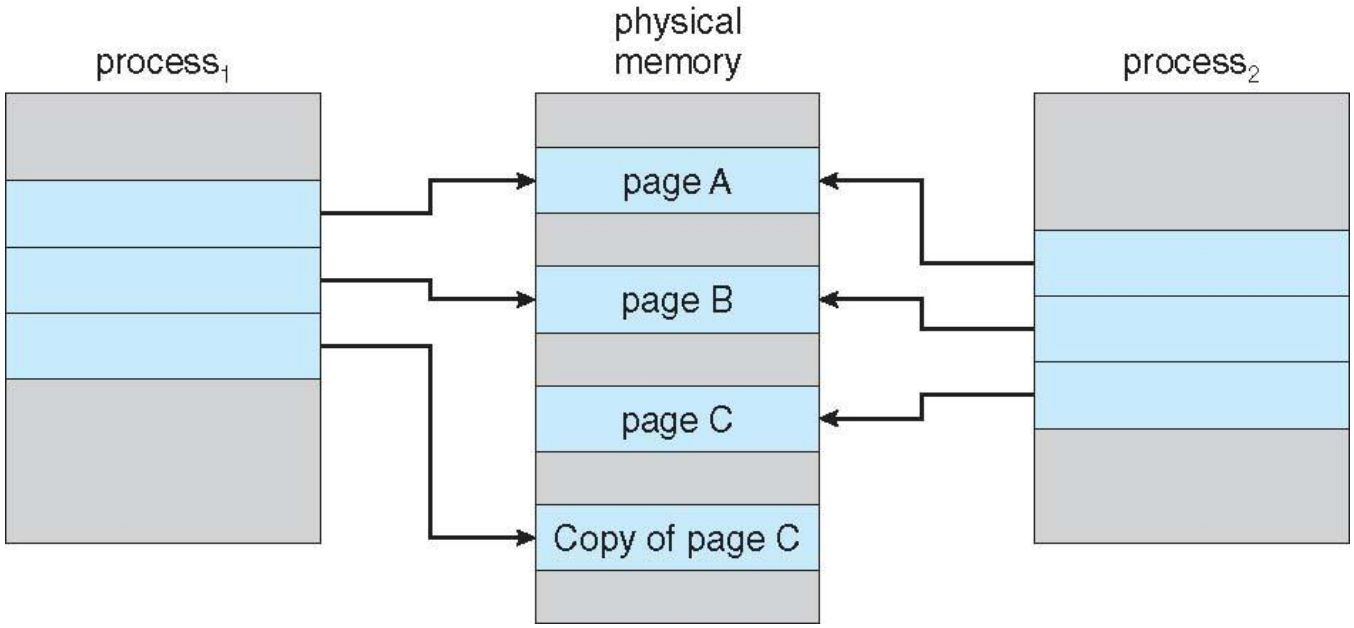


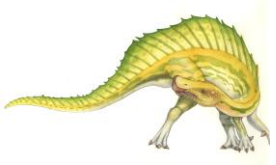
Before Process 1 Modifies Page C





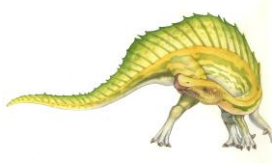
After Process 1 Modifies Page C





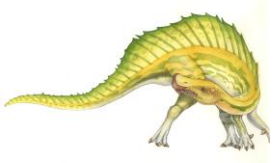
Page Replacement -Background

- Each page faults at most once,when it is first referenced.
- If a process of ten pages actually uses only half of them, then demand paging saves the I/O necessary to load the five pages that are never used.
- Could also increase our degree of multiprogramming by running twice as many processes
- With forty frames, run eight processes, rather than the four that could run if each required ten frames (five of which were never used).
- If we increase our degree of multiprogramming, we are **over-allocating** memory.
- If we run six processes, each of which is ten pages in size but actually uses only five pages, we have higher CPU utilization and throughput, with ten frames to spare.
- It is possible, however, that each of these processes, for a particular data set, may suddenly try to use all ten of its pages, resulting in a need for sixty frames when only forty are available



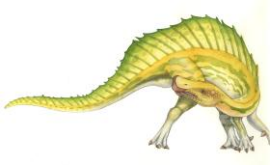
Page Replacement –Background Contd..

- system memory is not used only for holding program
- pages. Buffers for I/O also consume a considerable amount of memory. This use
- can increase the strain on memory-placement algorithms Deciding how much memory to allocate to I/O and how much to program pages is a significant challenge.
- Some systems allocate a fixed percentage of memory for I/O buffers, whereas others allow both user processes and the I/O subsystem to compete for all system memory.
- Over-allocation of memory manifests itself as follows.
- While a user process is executing, a page fault occurs.
- The operating system determines where the desired page is residing on the disk but then finds that there are *no* free frames on the free-frame list; all memory is in use



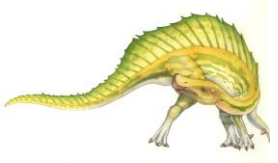
What Happens if There is no Free Frame?

- The operating system has several options
  - Terminate the user process.
    - demand paging is the operating system's attempt to improve the computer system's utilization and throughput.
    - Users should not be aware that their processes are running on a paged system—paging shouldbe logically transparent to the user. So
    - not the best choice
- The operating system could instead swap out a process, freeing all its frames and reducing the level of multiprogramming.
  - good one in certain circumstances

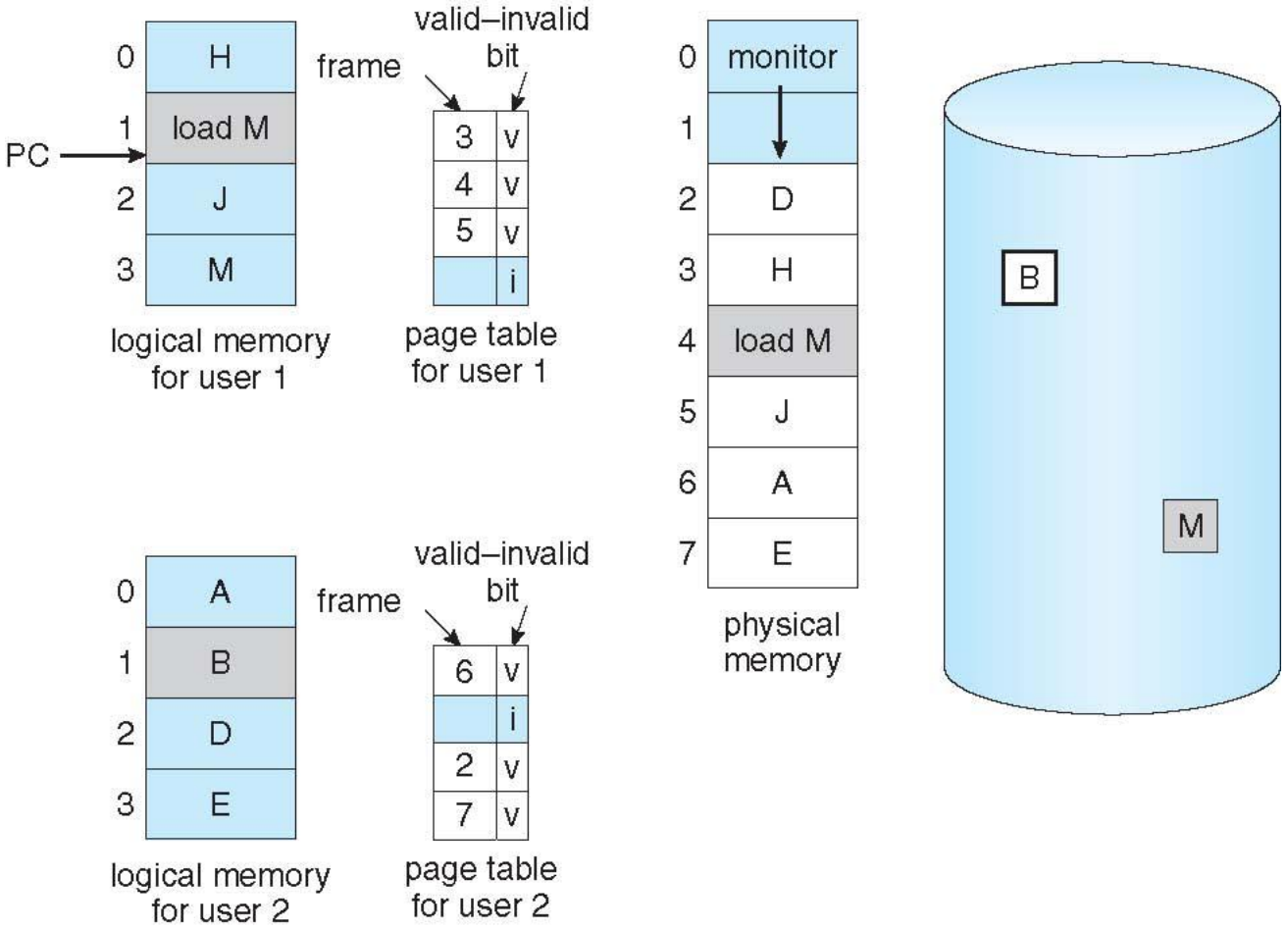


Basic Page Replacement

- If no frame is free, find one that is not currently being used and free it.
- can free a frame by writing its contents to swap space and changing the page table (and all other tables) to indicate that the page is no longer in memory
- can now use the freed frame to hold the page for which the process faulted.
- We modify the page-fault service routine to include page replacement:



Need For Page Replacement



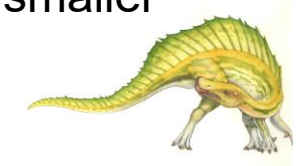
Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame:
   - If there is a free frame, use it
   - If there is no free frame, use a page replacement algorithm to select a **victim frame**
     - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

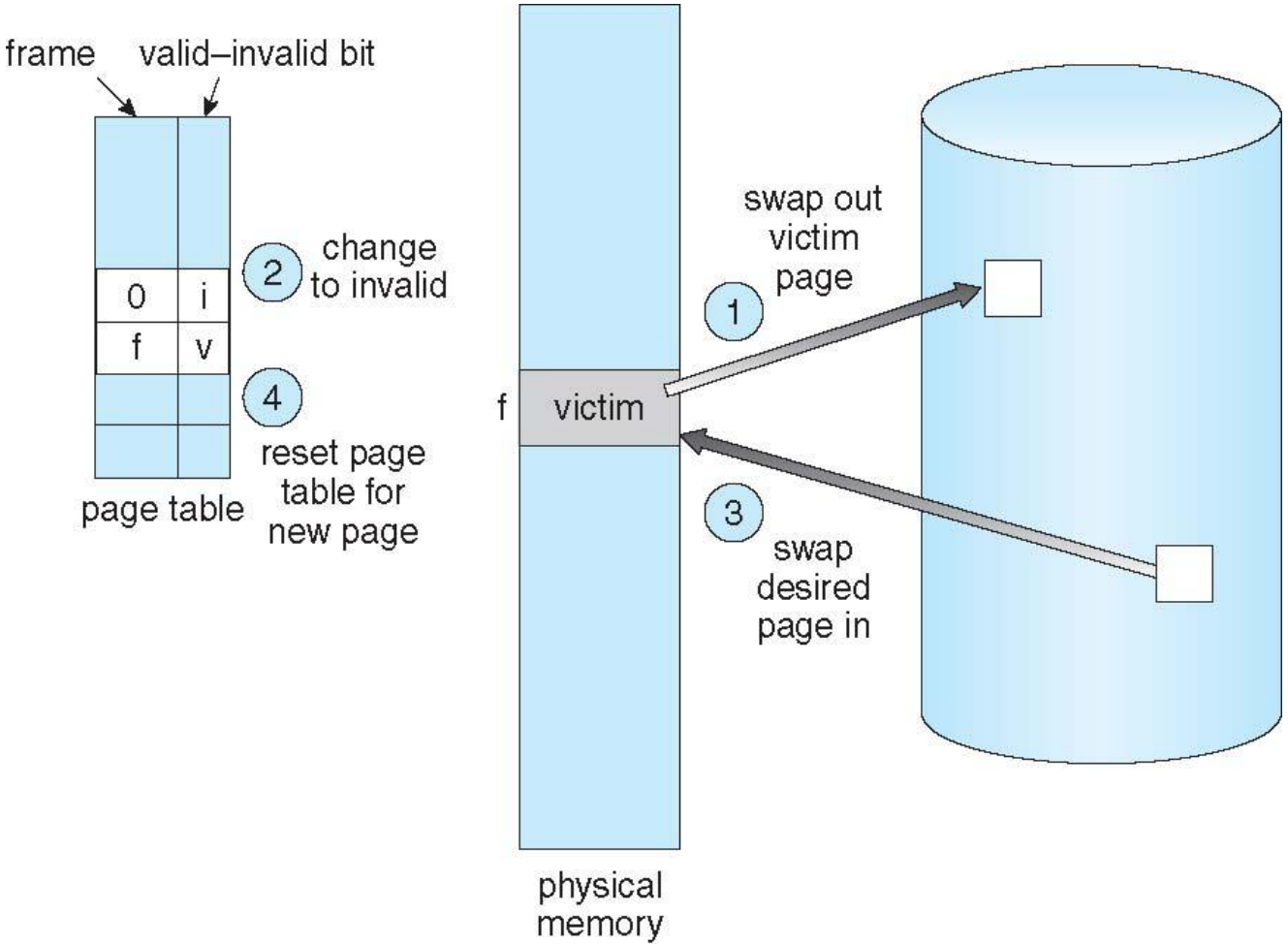
Note now **potentially 2 page transfers** for page fault – increasing EAT

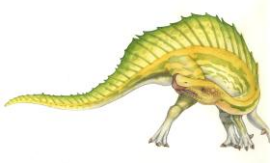
Use **modify** (**dirty**) **bit** to reduce overhead of page transfers – only modified pages are written to disk

Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory



Page Replacement

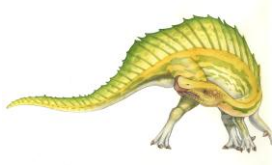




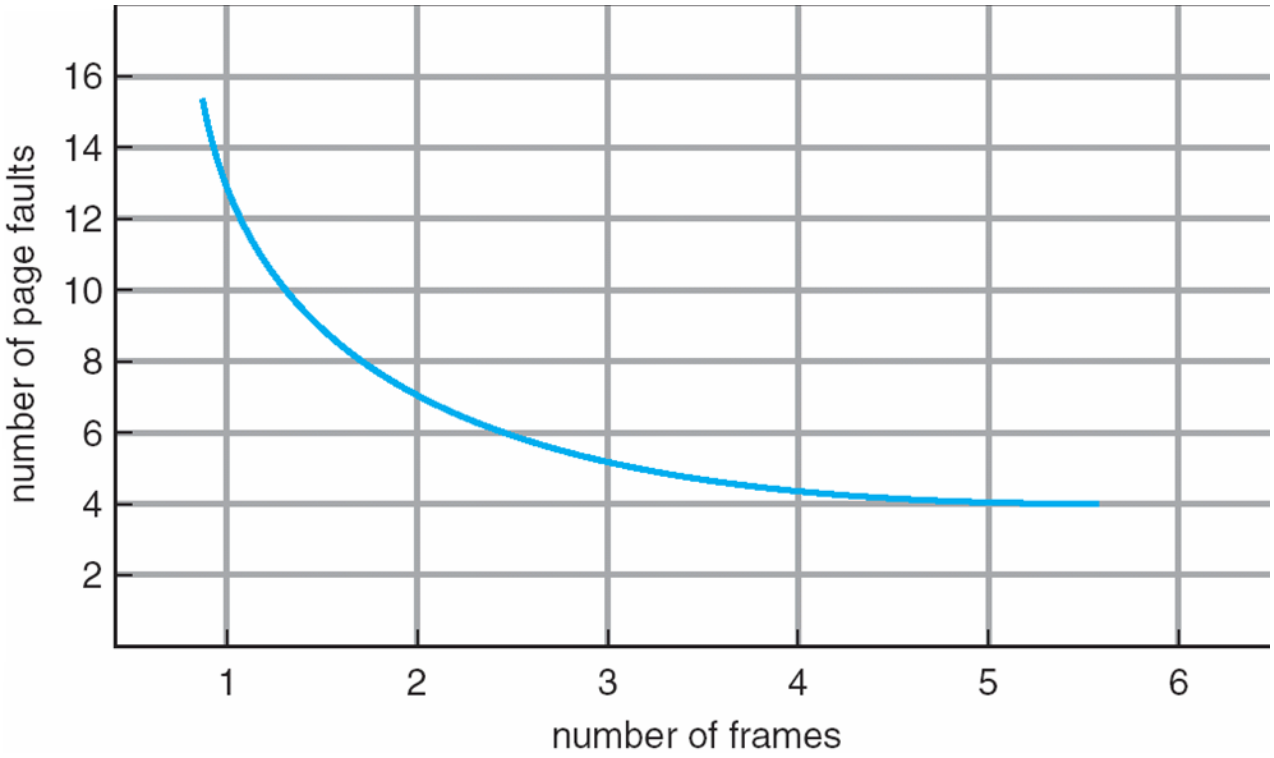
Page and Frame Replacement Algorithms

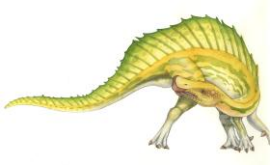
- **Frame-allocation algorithm** determines
  - How many frames to give each process
- **Page-replacement algorithm**
  - Which frames to replace
- Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
  - String is just page numbers, not full addresses
  - Repeated access to the same page does not cause a page fault
- Results depend on number of frames available - as the number of frames available increases, the number of page faults decreases.
- In all our examples, the **reference string** of referenced page numbers is

7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1



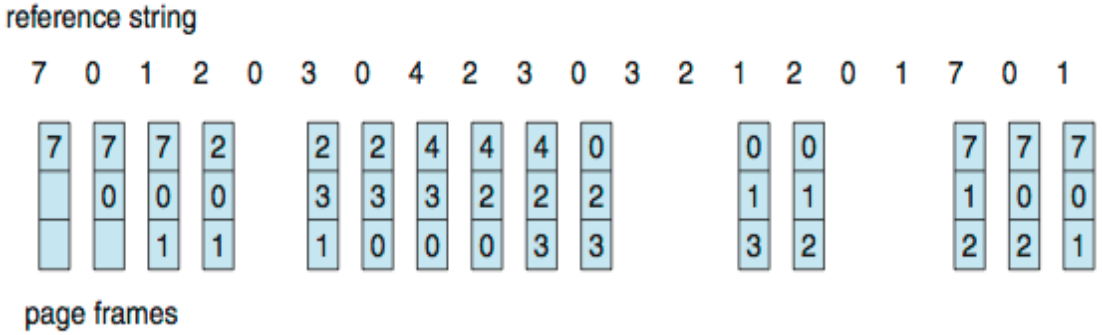
Graph of Page Faults Versus The Number of Frames





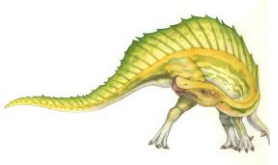
First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

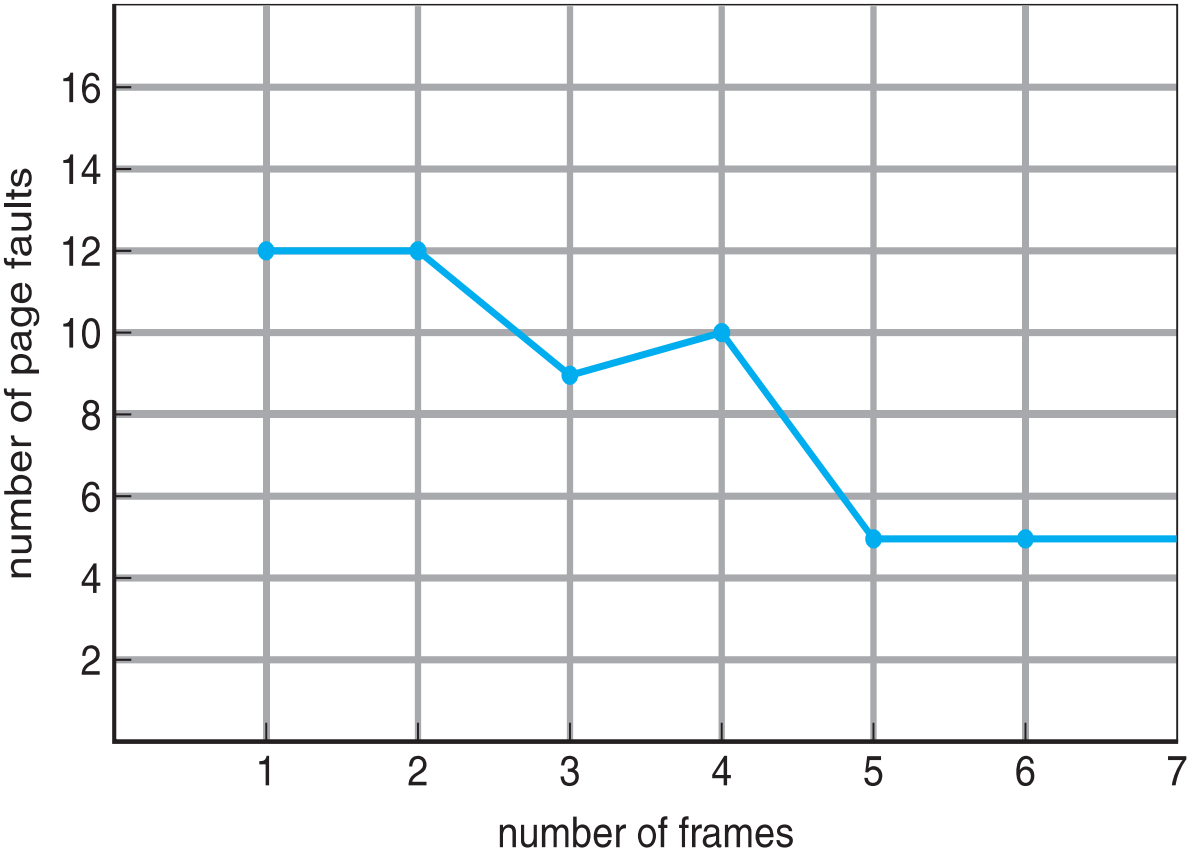


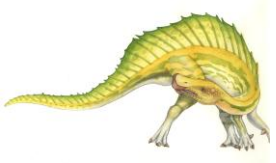
15 page faults

- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
  - Adding more frames can cause more page faults!
    - **Belady's Anomaly**
- How to track ages of pages?
  - Just use a FIFO queue



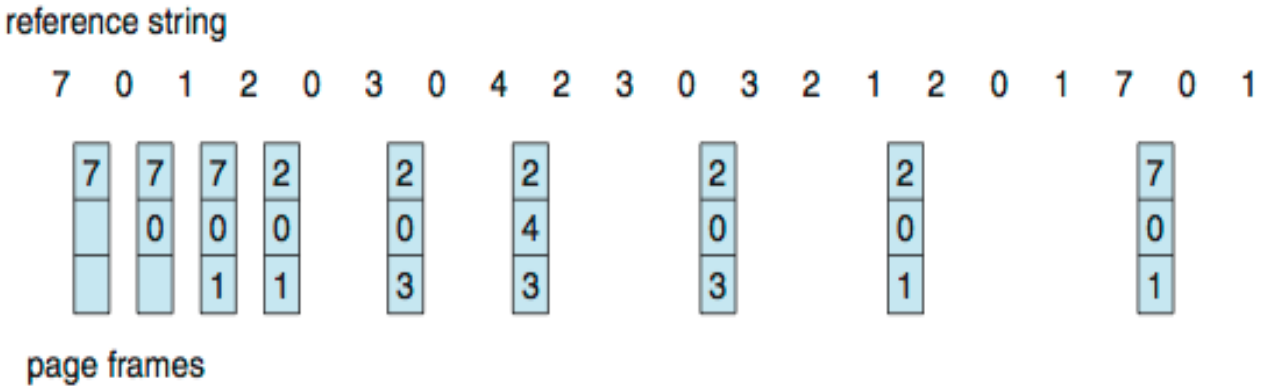
FIFO Illustrating Belady's Anomaly

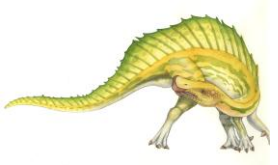




Optimal Algorithm

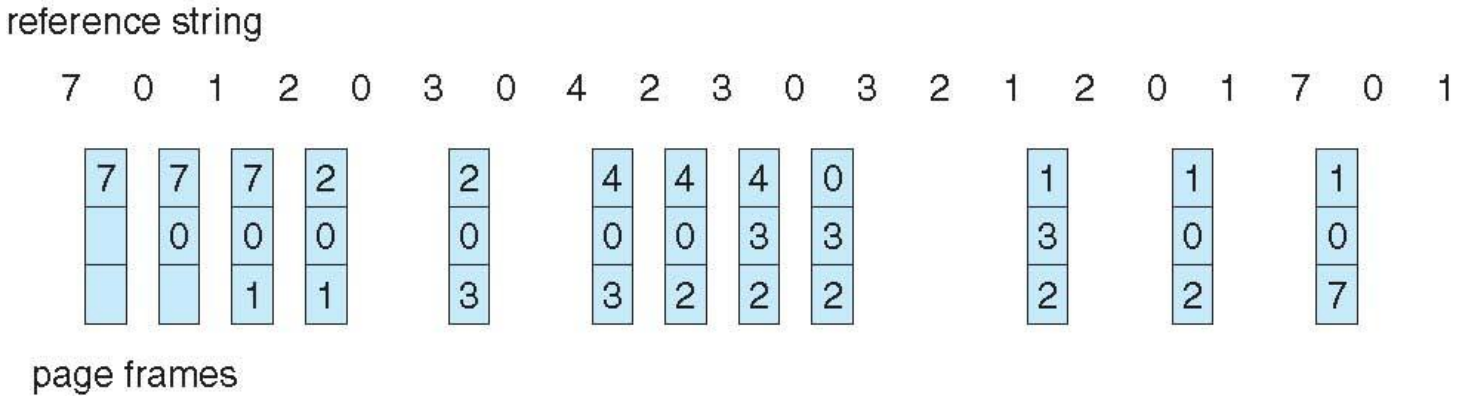
- Replace page that will not be used for longest period of time
  - 9 is optimal for the example
- How do you know this?
  - Can't read the future
- Used for measuring how well your algorithm performs





Least Recently Used (LRU) Algorithm

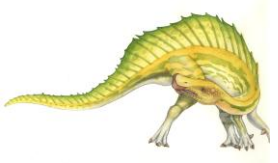
- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



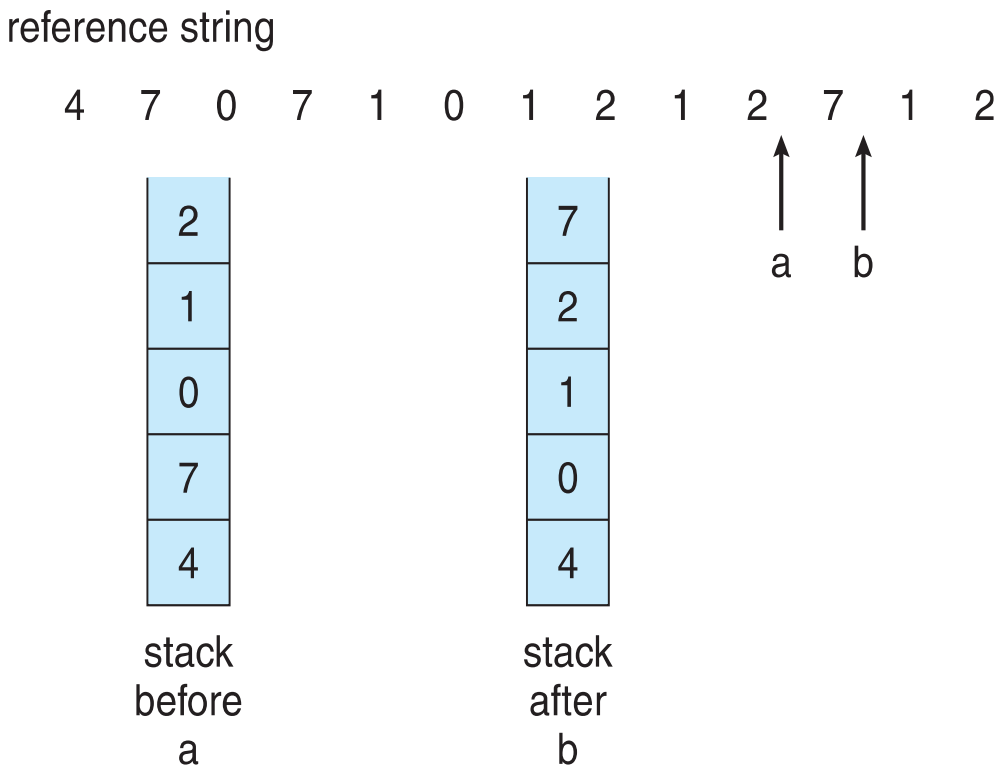
- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

LRU Algorithm (Cont.)

- Counter implementation
  - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to find smallest value
    - Search through table needed
- Stack implementation
  - Keep a stack of page numbers in a double link form:
  - Page referenced:
    - move it to the top
    - requires 6 pointers to be changed
  - But each update more expensive
  - No search for replacement
- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly

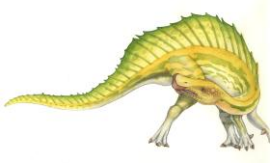


Use Of A Stack to Record Most Recent Page References



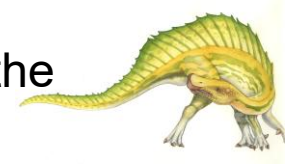
LRU Approximation Algorithms

- LRU needs special hardware and still slow
- **Reference bit**
  - With each page associate a bit, initially = 0
  - When page is referenced bit set to 1
  - Replace any with reference bit = 0 (if one exists)
    - We do not know the order, however

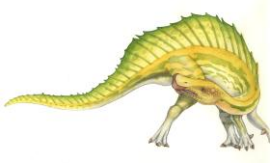


Additional-Reference-Bits Algorithm

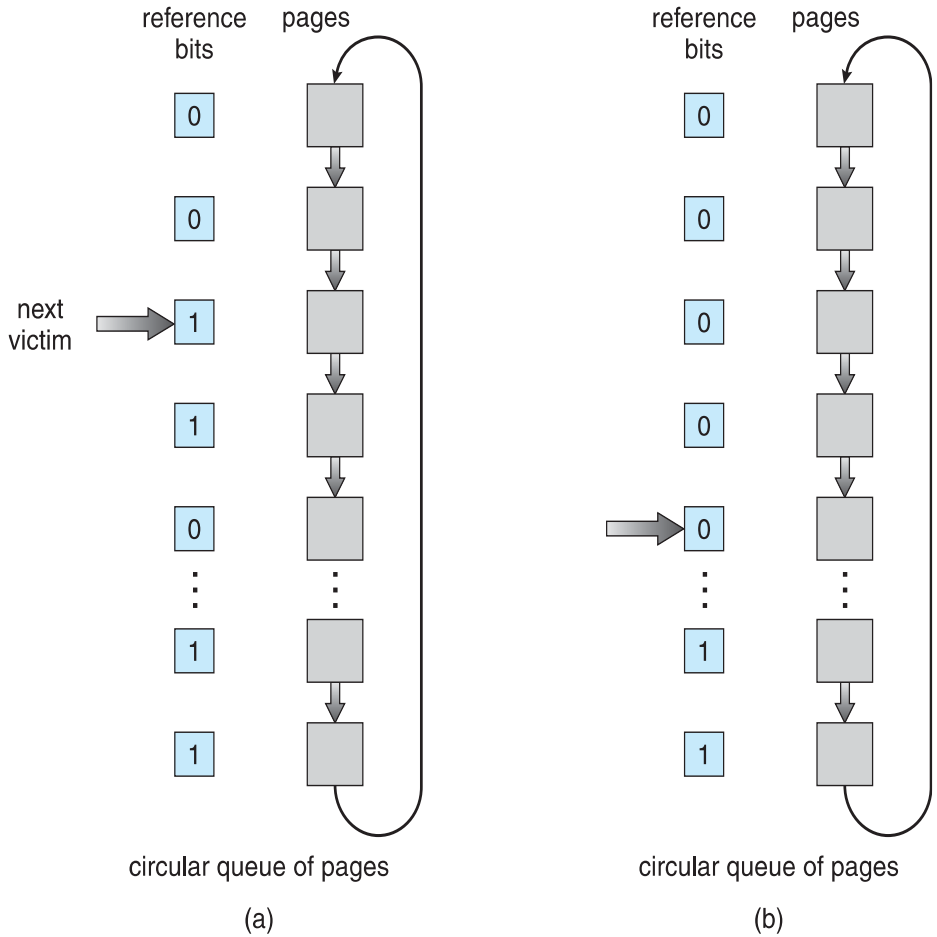
- ~~8-bit byte for each page in a table in memory.~~
- At regular intervals (say, every 100 milliseconds), a timer interrupt transfers
- control to the operating system.
- The operating system shifts the reference bit for each page into the high-order bit of its 8-bit byte, shifting the other bits right by 1 bit and discarding the low-order bit.
- These 8-bit shift registers contain the history of page use for the last eight time periods.
- If the shift register contains 00000000, for example, then the page has not been used for eight time periods;
- a page that is used at least once in each period has a shift register value of 11111111.
- A page with a history register value of 11000100 has been used more
- recently than one with a value of 01110111.
- interpret these 8-bit bytes as unsigned integers, the page with the lowest number is the LRU page, and it can be replaced.
- not guaranteed to be unique, either replace (swap out) all pages with the smallest value or use the FIFO method to choose among them



- **Second-chance algorithm**
  - Generally FIFO, plus hardware-provided reference bit
  - **Clock** replacement
  - If page to be replaced has
    - Reference bit = 0 -> replace it
    - reference bit = 1 then:
      - set reference bit 0, leave page in memory
      - replace next page, subject to same rules



Second-Chance (clock) Page-Replacement Algorithm

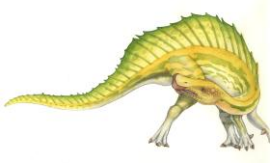


Enhanced Second-Chance Algorithm

- Improve algorithm by using reference bit and modify bit (if available) as an ordered pair
- Take ordered pair (reference, modify)

1. (0, 0) neither recently used not modified – best page to replace
2. (0, 1) not recently used but modified – not quite as good, must write out before replacement
3. (1, 0) recently used but clean – probably will be used again soon
4. (1, 1) recently used and modified – probably will be used again soon and need to write out before replacement

- When page replacement called for, use the clock scheme but use the four classes replace page in lowest non-empty class
  - Might need to search circular queue several times
- The major difference between this algorithm and the simpler clock algorithm is that here we give preference to those pages that have been modified to reduce the number of I/Os required.

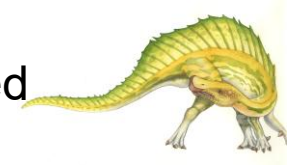


Counting Based Page Replacement

- Keep a counter of the number of references that have been made to each page and develop the following two schemes
  - **Least Frequently Used** (**LFU**) **Algorithm**: replaces page with smallest count
    - actively used page should have a large reference count.
    - A problem arises, -when a page is used heavily during the initial phase of a process but then is never used again.
    - Since it was used heavily-it has a large count and remains in memory even though it is no longer needed.
    - One solution is to shift the counts right by 1 bit at regular intervals, forming an exponentially decaying average usage count
  - **Most Frequently Used** (**MFU**) **Algorithm**: based on the argument that the page with the smallest count was probably just brought in and has yet to be used
- neither MFU nor LFU replacement is common.
- implementation of these algorithms is expensive, and they do not approximate OPT replacement well.

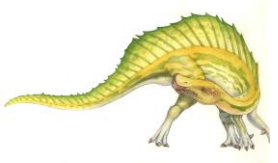
Page-Buffering Algorithms

- Keep a pool of free frames, always
  - Then frame available when needed, not found at fault time
  - Read page into free frame and select victim to evict and add to free pool
  - When convenient, evict victim
- Possibly, keep list of modified pages
  - When backing store or paging device is idle - write pages there and set to non-dirty
- Possibly, keep a pool of free frames and to remember which page was in each frame.
  - If referenced again before reused, no need to load contents again from disk
  - No I/O is needed I
  - When a page fault occurs, first check whether the desired page is in the free-frame pool.
  - If it is not, we must select a free frame and read into it
  - Generally useful to reduce penalty if wrong victim frame selected



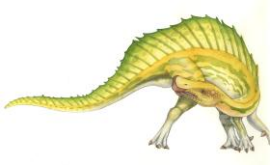
Allocation of Frames

- Each process needs ***minimum*** number of frames
- ***Maximum*** of course is total frames in the system
- as the number of frames allocated to each process decreases, the page-fault rate increases, slowing process execution.
- when a page fault occurs before an executing instruction is complete, the instruction must be restarted.
- must have enough frames to hold all the different pages that any single instruction can reference.
- the minimum number of frames per process is defined by the architecture,
- the maximum number is defined by the amount of available physical memory.



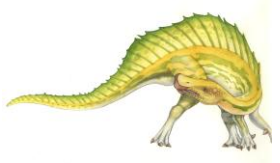
Allocation Algorithms

- Two major allocation schemes
  - **equal allocation**
  - Proportional allocation
- Many variations



Allocation Algorithms-Equal Allocation

- **Equal allocation** – easiest way to split *m* frames among *n* processes is to give everyone an equal share, *m*/*n* frames.
  - if there are 93 frames and five processes, each process will get 18 frames.
  - The three leftover frames can be used as a free-frame buffer pool.
- Consider a system with a 1-KB frame size.
- If a small student process of 10 KB and an interactive database of 127 KB are the only two processes running in a system with 62 free frames, it does not make much sense to give each process 31 frames.
- The student process does not need more than 10 frames, so the other 21 are wasted.
- Proportional allocation – Allocate according to the size of process
  - Dynamic as degree of multiprogramming, process sizes change



Allocation Algorithms- Proportional allocation

Proportional allocation -allocate available memory to each process according to its size.

Let the size of
the virtual memory for process pi be si , and define

s_i = size of process p_i

$S = \sum s_i$

m = total number of frames

a_i = allocation for $p_i = \frac{s_i}{S} \times m$

$m = 64$

$s_1 = 10$

$s_2 = 127$

$a_1 = \frac{10}{137} \square 62 \square 4$

$a_2 = \frac{127}{137} \square 62 \square 57$

Must adjust each ai to be an integer that is greater than the minimum number of frames required by the instruction set, with a sum not exceeding m.

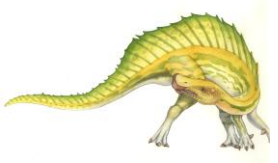
In both equal and proportional allocation, of course, the allocation may vary according to the multiprogramming level.

If the multiprogramming level is increased, each process will lose some frames to provide the memory needed for the new process.

Conversely, if the multiprogramming level decreases, the frames that were allocated to the departed process can be spread over the remaining processes.

Allocation Algorithms - Contd..

- with either equal or proportional allocation, a high-priority process is treated the same as a low-priority process.
- may want to give the high-priority process more memory to speed its execution, to the detriment of low-priority processes.
- One solution is to use a proportional allocation scheme wherein the ratio of frames depends not on the relative sizes of processes but rather on the priorities of processes or on a combination of size and priority.

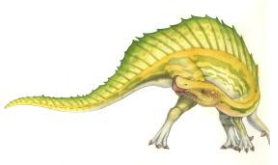


Global vs. Local Allocation

- **Global replacement** – process select a replacement frame from the set of all frames, even if that frame is currently allocated to some other process; that is, one process can take a frame from another.
- allow high-priority processes to select frames from low-priority processes for replacement.
- A process can select a replacement from among its own frames or the frames of any lower-priority process.
  - allows a high-priority process to increase its frame allocation at the expense of a low-priority process
  - process cannot control its own page-fault rate -set of pages in memory for a process depends not only on the paging behavior of that process but also on the paging behavior of other processes.
  - process execution time can vary greatly
  - a process may happen to select only frames allocated to other processes, thus increasing the number of frames allocated to it
  - But greater throughput so more common

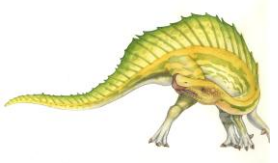
Global vs. Local Allocation Contd..

- **Local replacement** – each process selects from only its own set of allocated frames
  - More consistent per-process performance
  - But possibly underutilized memory

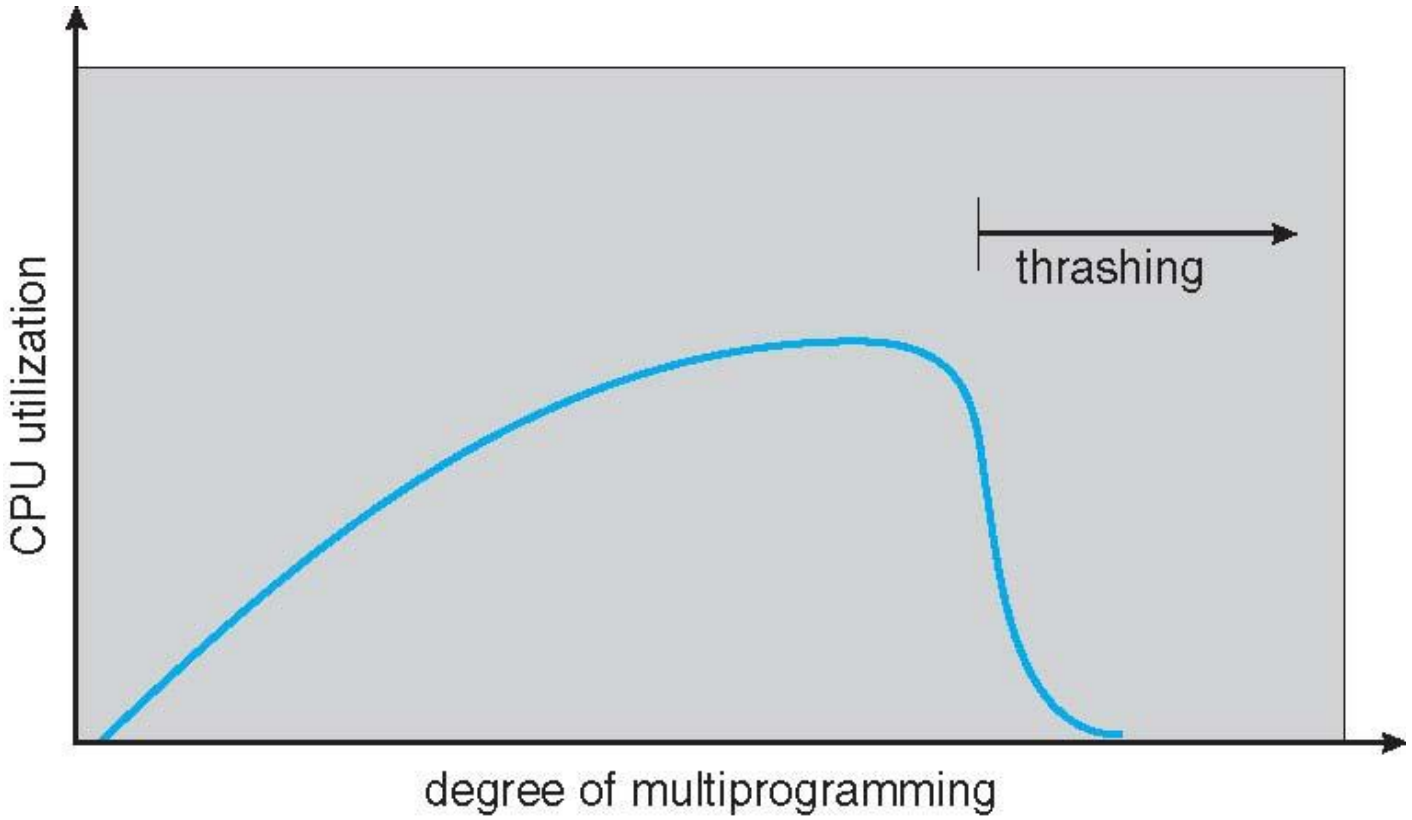


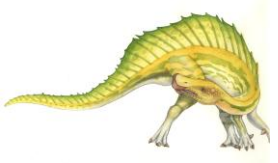
Thrashing

- If the number of frames allocated to a low-priority process falls below the minimum number required by the computer architecture, we must suspend that process's execution.
- Should then page out its remaining pages, freeing all its allocated frames.
- introduces a swap-in, swap-out level of intermediate CPU scheduling.
- If a process does not have "enough" pages, the page-fault rate is very high
- it must replace some page.
- However, since all its pages are in active use, it must replace a page that will be needed again right away.
- Consequently, it quickly faults again, and again, and again, replacing pages that it must bring back in immediately.
- This high paging activity is called **thrashing**.
- A process is thrashing if it is spending more time paging than executing.



Thrashing (Cont.)





End of Chapter 9